App Dev 1 Project Report

1. Student Details

- **Name:** Tejashvi Yadav
- **Roll Number:** 24f2004841

  I'm a student right now, and I'm working on getting my degree. Before I started this project, I was a little scared of how complicated database administration, authentication, and backend web development are. But I was able to build this platform from scratch by being persistent, reading the documentation, and trying things out in the real world. This very rewarding educational experience has helped me learn more about Flask, secure routing, and relational databases in a practical way.

2. Project Details

- **Project Title:** HIreable.
- **Problem Statement:** To design and build a comprehensive web-based platform that streamlines the hiring process. The platform must connect students looking for jobs with companies hosting hiring drives while providing administrators with powerful oversight tools. The system must address key challenges such as data isolation, role-based access control, and platform integrity.
- **Approach:** The application was built using Flask as the backend framework with SQLAlchemy ORM for database management. The system implements a secure, modular architecture utilizing Role-Based Access Control (RBAC), supporting three distinct user roles: admin, company, and student. Key features include dynamic hiring drive creation, application tracking, profile management, and an admin dashboard for centralized oversight.

3. AI/LLM Declaration

I utilized AI tools (like ChatGPT/Gemini) primarily as a learning aid and debugging assistant. Specifically, I used them to:

- Implement the search feature in the company dashboard.

The AI assistance was limited to roughly 10-15% of the project workflow.

4. Technologies and Frameworks Used

Technology/Library	Purpose
Flask	Core backend web framework for routing, blueprints, and request handling.
SQLAlchemy	ORM for database modeling and relational integrity.
Jinja2	Template engine for rendering dynamic HTML UI.
SQLite	Lightweight relational database for data persistence.
HTML/CSS	Frontend templating and skeleton..
Flask-Login	User authentication, RBAC, and session handling.
Bootstrap	For responsive UI/UX using classes.

5. Database Schema / ER Diagram

Tables:

- **User:** Stores core credentials and RBAC logic (id, username, name, password_hash, role, status).
- **Student:** Extended details for students (id, name, user_id, age, department, skills).
- **Company:** Extended details for companies (id, user_id, name, hr_contact, website, status).
- **Drive:** Hiring events hosted by companies (id, date, title, description, salary, eligibility, deadline, status, company_id).
- **Application:** Bridge table linking students to drives (id, drive_id, student_id, description, date, status).

Key Relationships:

- **One-to-One:** User <-> Student / User <-> Company.
- **One-to-Many:** Company -> Drive (Companies host multiple drives).
- **One-to-Many:** Drive -> Application (A drive receives multiple applications).
- **One-to-Many:** Student -> Application (A student makes multiple applications).

6. API Resource Endpoints

Endpoint	Method	Description
/student_register	GET/POST	Register a new student and create a profile.
/company_register	GET/POST	Register a new company (Status defaults to pending).
/[role]_login	GET/POST	Authenticate user and route to role-specific dashboard.
/dashboard	GET/POST	Student dashboard showing active companies and applied drives.
/student/apply/<drive_id>	POST	A student applies to a specific active hiring drive.
/company/new_drive	GET/POST	The company creates a new hiring drive listing.
/company/drive/<id>/application	GET/POST	The company views student applications for a specific drive.
/company/update_application/<id>	POST	The company updates a student's application status.
/admin_dashboard	GET/POST	Admin dashboard with system metrics and global search.
/blacklist/<user_id>	POST	Admin blacklists a user, cascading deletion of associated data.

7. Architecture and Features

Architecture Overview & Implemented Features:

- **Authentication & Authorization:** Secure login with Werkzeug password hashing. Route protection implemented using @login_required and strict current_user.role checks across Blueprints.
- **Company Drive Management:** Companies can create drives, set duration/deadlines, post salaries, and close drives. Strict ownership queries (current_user.company.id == Drive.company_id) ensure companies only view and edit their own data.
- **Student Application System:** Students can browse active companies, view drive details, and submit applications. Database constraints (UniqueConstraint('drive_id', 'student_id')) inherently prevent duplicate applications.
- **Admin Oversight System:** Admins can approve or reject pending companies, monitor job applications, manage all active/closed drives, and blacklist malicious actors.

8. Video Presentation

- **DriveLink: https://drive.google.com/file/d/1xtcv-b8Oh4HQN6HnKJ-ef_bgx3R7pv5M/view?usp=drive_link**
- **Video Duration:** Approximately 5-8 minutes.
- **Demonstrated Content:** The video showcases the complete end-to-end workflow of the platform. It begins with the registration of a new company and the subsequent admin approval process. It then highlights a student registering, browsing active hiring drives, and submitting an application. Finally, it covers the company's perspective of reviewing applications and updating their status, concluding with a walkthrough of the admin dashboard's global search and user management functionalities.

9. Key Implementation Highlights

- **Authorization Security:** Implemented conditional logic on protected routes to verify current_user.role before rendering templates or executing sensitive logic.
- **Data Integrity:** Utilized SQLAlchemy cascade="all, delete-orphan" extensively on the User models. For example, blacklisting and deleting a company automatically wipes all active drives and pending applications associated with them to keep the database clean.
- **Advanced Global Search:** Built an admin search feature utilizing SQLAlchemy's or_ operator and cast() function to search simultaneously across text fields (company/student names) and integer fields (user IDs).

10. Testing & Validation

Tested Scenarios:

- Attempted to force-navigate to /admin_dashboard or /company_dashboard while logged in as a Student (Successfully blocked and redirected with flash messages).
- Registered a new user with an already existing username (Triggered unique constraint and handled via error UI).
- Verified that a company can only view and update applications for drives they have

explicitly created (Cross-tenant data access blocked).
- Tested the Admin's blacklist functionality to ensure that when a company is blacklisted, all their associated active drives and student applications are dynamically deleted.

11. Submission Contents

The ZIP file includes:

1. **Project Report.pdf** - This report document.
2. **app.py** - Main Flask application module and entry point.
3. **models.py** - Database models schema and relationships.
4. **routes/** - Folder containing blueprints for admin_routes.py, student_routes.py, and company.py.
5. **templates/** - All Jinja2 HTML templates for rendering views.
6. **assets/** - Static folder for custom styling and assets.
7. **requirements.txt** - Python dependencies.
8. **README.md** - Instructions to run the project.

12. How to Run the Project

Requirements:

- Python 3.8+
- Flask, Flask-SQLAlchemy, Flask-Login, Werkzeug

Steps:

1. Extract the ZIP file and navigate to the root directory.
2. Create and activate a virtual environment (python -m venv venv / source venv/bin/activate).
3. Install dependencies: pip install -r requirements.txt
4. Start the server: python app.py (The SQLite database and default Admin user are created automatically upon startup).
5. Access the application at http://127.0.0.1:5000